

# 辅助结构与图算法

算法目标怎样调用辅助结构推进不变量

408 · 数据结构 Internal Bridge B03

桥梁接口

Algorithm Needs Operation → Data Structure Provides Operation → Candidate State Changes → Algorithm Invariant

Heap 与 Union-Find 不因为被图算法调用就失去自身 Owner；图算法只获得优先队列的极值操作或并查集的连通判定。

职责分离

DS05 Owns 上滤、下滤和堆序；DS06 Owns Find、Union 和路径压缩；DS08 Owns cut/relax/入度等全局正确性。调用方只依赖稳定接口，不复制底层实现。

## 目录

|   |        |   |
|---|--------|---|
| 1 | 三个交接例子 | 2 |
| 2 | 最小调用契约 | 2 |
| 3 | 做题控制协议 | 2 |

## 1 三个交接例子

Prim 需要从跨越当前树边界的候选边中取最小值；Dijkstra 需要从暂定距离中取最小值并允许新候选入队；Kruskal 需要判断边两端是否已在同一分量，并在安全边加入后合并。三个算法都可使用辅助结构，但“何时安全”不是由 Heap/Union-Find 决定。

| 算法       | 调用接口            | 算法不变量          | 失败/边界   |
|----------|-----------------|----------------|---------|
| Prim     | push/top/pop    | 已选顶点集的 cut 安全边 | 非连通图    |
| Dijkstra | push/top/pop    | 非负权下取出距离已确定    | 负边      |
| Kruskal  | connected/unite | 已选边无环且逐步扩展森林   | 重复边/非连通 |

## 2 最小调用契约

```
1 while (!pending.empty()) {
2   Candidate c = pending.pop_min();
3   if (algorithm_invariant_allows(c)) commit(c);
4   else discard(c);
5   add_new_candidates(c);
6 }
```

Kruskal 的对应动作是 ‘if (!connected(u,v)) unite(u,v); commit(edge);’。这里的 ‘commit’ 仍由 DS08 证明；本桥只规定调用方向和信息边界。

## 3 做题控制协议

先圈算法全局目标，再圈辅助结构实际提供的一个操作；将“辅助结构内部复杂度”和“算法候选状态数量”分开相乘。若解答把完整堆代码或并查集递归塞进图算法证明，删回接口级说明并补全算法不变量。

压缩复原

三问：算法要什么操作？辅助结构保证什么？这个操作怎样让全局不变量前进一步？